

Chapter 4: Local Descent

Algorithms for Optimization, 2nd ed. (2026)

Kochenderfer & Wheeler

MIT Press

Chapter Outline

- 1 4.1 Descent Direction Iteration
- 2 4.2 Step Factors
- 3 4.3 Line Search
- 4 4.4 Approximate Line Search — Wolfe Conditions
- 5 4.5 Trust Region Methods
- 6 4.6 Termination Conditions
- 7 4.7 Summary & Exercises

Motivation: What Problem Are We Solving? I

In this chapter we tackle the central problem of applied optimization: find a point $x^* \in \mathbb{R}^n$ (read: “x-star in R-n”, meaning an n -dimensional real-valued vector) that minimises a scalar objective function $f : \mathbb{R}^n \rightarrow \mathbb{R}$.

Real-world functions are typically *non-convex* and possess many local minima, so there is no algebraic shortcut that solves everything at once. Instead we use an *iterative* strategy: start from an initial guess and repeatedly move the design point in a direction that makes f smaller, stopping when progress becomes negligibly small.

The Core Idea of Local Descent

Local descent methods build a sequence of points

$$x^{(0)}, x^{(1)}, x^{(2)}, \dots$$

(read: “x superscript 0, x superscript 1, ...”, where the superscript is the iteration number) such that $f(x^{(k+1)}) \leq f(x^{(k)})$ at every step.

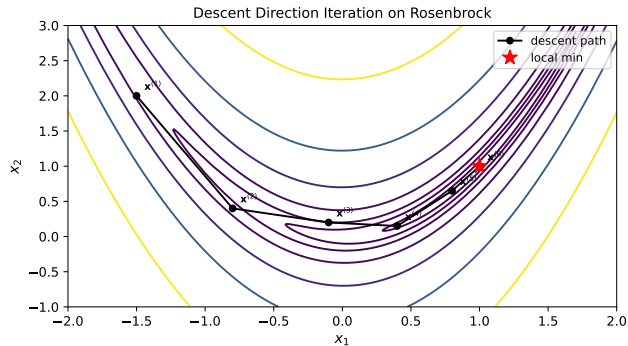
Motivation: What Problem Are We Solving? II

Notation

- $\mathbf{x}^{(k)}$ — the current design point at iteration k . This is a vector of n real numbers (e.g. the coordinates of a point in 2-D space if $n = 2$).
- $f(\mathbf{x}^{(k)})$ — the value of the objective function evaluated at the current point; this is a single real number.
- $\mathbb{R}^n$ — the set of all n -dimensional real-valued vectors.

Motivation: What Problem Are We Solving? III

The figure below illustrates how successive iterates trace a path downhill across the objective landscape. Each arrow represents one step of the algorithm — a carefully chosen direction combined with a carefully chosen step size.



Motivation: What Problem Are We Solving? IV

Key Insight

The challenge is not whether to move downhill — that is easy — but *how far* to move (the step size) and in *exactly which downhill direction* (the descent direction). Different answers give methods with very different convergence speeds and computational costs, as we will see throughout this chapter.

The Four-Step Template of Every Descent Method I

Nearly every gradient-based optimization algorithm in use today is a specialisation of the following four-step template. Understanding this template deeply makes it straightforward to analyse any new method you encounter.

Descent Direction Iteration (Algorithm 4.1)

- 1 **Check convergence.** Evaluate the local gradient $\nabla f(x^{(k)})$ (read: “nabla f of x-k”, where ∇ is the nabla symbol representing the gradient, i.e. the vector of all partial derivatives). If the gradient is sufficiently small — meaning the surface is nearly flat — stop and report $x^{(k)}$ as the solution.
- 2 **Choose a descent direction $d^{(k)}$.** The vector $d^{(k)} \in \mathbb{R}^n$ (read: “d superscript k”) must point in a direction along which f decreases.
- 3 **Choose a step factor $\alpha^{(k)} > 0$.** The scalar $\alpha^{(k)}$ (alpha, pronounced “al-fa”, the first letter of the Greek alphabet) controls how far we move in the chosen direction.
- 4 **Update the design point:**

$$x^{(k+1)} = x^{(k)} + \alpha^{(k)} d^{(k)}.$$

The Four-Step Template of Every Descent Method II

Notation for the Update Rule

- $x^{(k+1)}$ — the *new* design point after the update.
- $x^{(k)}$ — the *current* design point before the update.
- $\alpha^{(k)}$ (alpha) — step factor (also called step size or learning rate in machine learning); a positive scalar that scales the length of the step.
- $d^{(k)}$ (bold d) — descent direction vector; it encodes *which way* to move.
- The product $\alpha^{(k)}d^{(k)}$ is the actual displacement added to the current point.

How to Read This Update Rule

Think of $x^{(k)}$ as your current location on a hillside. The vector $d^{(k)}$ is a compass direction pointing downhill. The scalar $\alpha^{(k)}$ is how many metres you walk in that direction. After walking, your new location is $x^{(k+1)}$. The art of local descent lies entirely in choosing $d^{(k)}$ and $\alpha^{(k)}$ wisely.

What Makes a Direction a “Descent” Direction? I

Before we can choose a descent direction we need to know precisely what the term means mathematically. The key tool is the *directional derivative*, which measures how fast f changes as we move from x in direction d .

Definition: Directional Derivative

The directional derivative of f at point x in direction d is

$$\nabla_d f(x) = \nabla f(x)^\top d,$$

where $\nabla f(x)$ (nabla f of x) is the gradient vector — the column vector of all partial derivatives of f evaluated at x — and the superscript $^\top$ denotes the transpose (turning a column vector into a row vector so that the dot product is well-defined).

What Makes a Direction a “Descent” Direction? II

Notation

- $\nabla_d f(x)$ — the directional derivative of f at x along direction d ; this is a single real number.
- $\nabla f(x)$ — the gradient of f at x ; an n -dimensional column vector.
- $^\top$ — transpose operator.
- $\nabla f(x)^\top d$ — the dot product (inner product) of the gradient with the direction vector; geometrically it equals $\|\nabla f\| \|d\| \cos \theta$ (theta, pronounced “thay-ta”), where θ is the angle between the two vectors.

Definition: Descent Direction

A vector $d \in \mathbb{R}^n$ is called a *descent direction* at x if and only if

$$\nabla f(x)^\top d < 0.$$

This condition means the directional derivative is *negative*, so moving any sufficiently small positive distance α along d from x will decrease the value of f .

What Makes a Direction a “Descent” Direction? III

Worked Example: Verifying a Descent Direction

Let $f(x_1, x_2) = x_1^2 + x_2^2$ (a simple bowl). At the point $x = [3, 4]^\top$ the gradient is

$$\nabla f(x) = [2x_1, 2x_2]^\top = [6, 8]^\top.$$

Is the direction $d = [-1, -1]^\top$ a descent direction?

$$\nabla f(x)^\top d = [6, 8] \cdot [-1, -1] = 6 \times (-1) + 8 \times (-1) = -14 < 0. \quad \checkmark$$

Yes — moving in the $[-1, -1]$ direction from $[3, 4]$ will decrease f .

The negative gradient direction $-\nabla f(x) = [-6, -8]^\top$ is also a descent direction (and in fact is the direction of *steepest* descent):

$$\nabla f(x)^\top (-\nabla f(x)) = -\|\nabla f(x)\|^2 = -(6^2 + 8^2) = -100 < 0. \quad \checkmark$$

What Makes a Direction a “Descent” Direction? IV

Key Insight

The gradient $\nabla f(x)$ points in the direction of *steepest ascent* (uphill). Therefore the *negative* gradient $-\nabla f(x)$ always points in the direction of steepest descent (downhill). This is the basis of gradient descent, the most widely used optimisation method in machine learning.

Three Important Descent Directions

- **Steepest descent:** $d = -\nabla f(x)$. Always a descent direction; easy to compute; may converge slowly.
- **Newton direction:** $d = -H^{-1}\nabla f(x)$, where H (bold H) is the Hessian matrix of second derivatives. Exploits curvature; converges much faster near the minimum but requires computing and inverting H .
- **Quasi-Newton directions:** approximate H^{-1} cheaply from gradient differences. A practical middle ground (covered in later chapters).

Algorithm 4.1 in Python: Generic Descent Iteration

```
1  import numpy as np
2
3  def descent_direction_iteration(f, grad_f, x0, direction_fn,
4                                step_fn, max_iter=1000, tol=1e-6):
5      """
6      Generic descent direction iteration.
7      Args:
8          f          : objective function  f(x) -> float
9          grad_f      : gradient function  grad_f(x) -> ndarray
10         x0          : initial design point (ndarray)
11         direction_fn: d = direction_fn(x, g) -> descent direction
12         step_fn      : alpha = step_fn(f, grad_f, x, d) -> step factor
13     Returns:
14         x          : converged design point
15         history    : list of all iterates
16     """
17     x = x0.copy().astype(float)
18     history = [x.copy()]
19     for k in range(max_iter):
20         g = grad_f(x)
21         if np.linalg.norm(g) < tol:      # gradient-magnitude termination
22             break
23         d = direction_fn(x, g)           # choose descent direction
24         alpha = step_fn(f, grad_f, x, d) # choose step factor
25         x = x + alpha * d               # update rule:  $x^{(k+1)} = x^{(k)} + \alpha * d$ 
26         history.append(x.copy())
27     return x, history
```

Step Factors: How Far Should We Move? I

Once we have chosen a descent direction $d^{(k)}$, the next decision is how far to walk along it. The *step factor* $\alpha^{(k)}$ (alpha, the first Greek letter, pronounced “al-fa”) is a positive scalar multiplying the direction vector. A step factor that is too large causes the algorithm to overshoot and may even diverge; one that is too small wastes iterations and slows convergence dramatically.

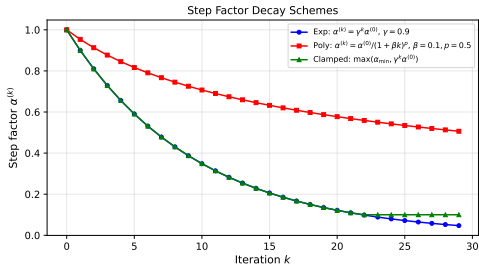
Fixed Step Factor

The simplest strategy is to use the same value α_0 at every iteration:

$$\alpha^{(k)} = \alpha_0 \quad \text{for all } k.$$

Here α_0 (alpha sub zero) is a constant chosen before the algorithm starts. This approach is easy to implement but requires careful tuning: if α_0 is too large the iterates diverge; if it is too small the method crawls.

A smarter strategy is to *decrease* the step factor over time. Early in the search, when we are far from the minimum, large steps help us explore. Later, when we are close, small steps allow fine-tuned convergence without



Step Factors: How Far Should We Move? II

Exponential Decay (Eq. 4.2)

$$\alpha^{(k)} = \gamma^k \alpha^{(0)}, \quad 0 < \gamma < 1.$$

At each iteration the step factor is multiplied by γ (gamma, the third Greek letter, pronounced “gam-a”), so after k steps it has been shrunk by a factor of γ^k . Since $0 < \gamma < 1$, this converges to zero exponentially fast.

Notation for Exponential Decay

- $\alpha^{(0)}$ (alpha superscript 0) — the initial step factor, i.e. the step size used on the very first iteration.
- γ (gamma) — the decay rate, a number strictly between 0 and 1, e.g. $\gamma = 0.9$ means each step is 90% of the previous one.
- γ^k (gamma to the power k) — the cumulative shrinkage after k iterations.

Step Factors: How Far Should We Move? III

How to Read This: Total Distance Bound

If the direction vectors have bounded length ($\|d^{(k)}\| \leq \delta$, delta pronounced “del-ta”), the total distance travelled by the algorithm is at most

$$\sum_{k=0}^{\infty} \alpha^{(k)} \|d^{(k)}\| \leq \delta \alpha^{(0)} \sum_{k=0}^{\infty} \gamma^k = \frac{\delta \alpha^{(0)}}{1 - \gamma}.$$

Here $\sum_{k=0}^{\infty}$ (Sigma, the capital Greek letter for “sum”) sums over all iterations from $k = 0$ to infinity. This geometric series converges because $\gamma < 1$, guaranteeing that the algorithm does not wander off to infinity even if it runs forever.

Step Factors: How Far Should We Move? IV

Polynomial Decay (Eq. 4.4)

$$\alpha^{(k)} = \frac{\alpha^{(0)}}{(1 + \beta k)^p}, \quad \beta > 0, p > 0.$$

Here β (beta, pronounced “bay-ta”) is a scaling constant that controls how quickly the denominator grows, and p (p) is the polynomial decay exponent.

Notation for Polynomial Decay

- β (beta) — positive constant; larger β means faster initial decay.
- p (p) — exponent; $p = 1$ gives harmonic decay ($\alpha^{(k)} \propto 1/k$), $p = 2$ gives faster decay.
- When $0 < p \leq 1$, the sum $\sum_k \alpha^{(k)}$ is *unbounded*, meaning the algorithm can in principle reach *any* point in space given enough iterations — a property required by stochastic gradient descent (SGD) theory to guarantee convergence to a global minimum.
- When $p > 1$, the sum is bounded and the algorithm is constrained to a finite region.

Step Factors: How Far Should We Move? V

Clamped Exponential Decay (Eq. 4.5)

$$\alpha^{(k)} = \max\left(\alpha_{\min}, \gamma^k \alpha^{(0)}\right).$$

Here $\alpha_{\min}$ (alpha sub min) is a floor value that prevents the step from shrinking all the way to zero. The max operator returns whichever of its two arguments is larger, so the step never drops below $\alpha_{\min}$. This avoids *premature stagnation*: the algorithm keeps making progress rather than grinding to a halt.

Step Factor Decay Schedules in Python

```
1 import numpy as np
2
3 def exponential_decay(alpha0, gamma, k):
4     """Exponential decay:  $\alpha^k = \gamma^k * \alpha_0$ ,  $0 < \gamma < 1$ ."""
5     return gamma**k * alpha0
6
7 def polynomial_decay(alpha0, beta, p, k):
8     """Polynomial decay:  $\alpha^k = \alpha_0 / (1 + \beta * k)^p$ ,  $\beta > 0$ ,  $p > 0$ ."""
9     return alpha0 / (1.0 + beta * k)**p
10
11 def clamped_decay(alpha0, gamma, alpha_min, k):
12     """Clamped decay:  $\alpha^k = \max(\alpha_{\min}, \gamma^k * \alpha_0)$ ."""
13     return max(alpha_min, gamma**k * alpha0)
14
15 # -- Worked numerical example -----
16 #  $f(x) = x^2$ , initial point  $x_0 = 3.0$ , exponential decay
17 def f(x): return x**2
18 def df(x): return 2 * x # gradient of  $f(x) = x^2$  is  $f'(x) = 2x$ 
19
20 x = 3.0
21 print(f"k=0: x={x:.4f}, f(x)={f(x):.4f}, grad={df(x):.4f}")
22 for k in range(10):
23     alpha = exponential_decay(alpha0=0.5, gamma=0.85, k=k)
24     x = x - alpha * df(x) #  $x_{k+1} = x_k - \alpha * f'(x_k)$ 
25     print(f"k={k+1}: alpha={alpha:.4f}, x={x:.4f}, f(x)={f(x):.6f}")
26     if abs(df(x)) < 1e-4:
27         print("Converged!")
```

Line Search: Finding the Best Step Along a Direction I

Once we have fixed the descent direction d , choosing the step factor $\alpha^{(k)}$ reduces to a one-dimensional problem: how far along the ray $x + \alpha d$ should we walk? This is called a *line search*.

Exact Line Search

Exact line search selects the α that minimises f along the ray:

$$\alpha^* = \arg \min_{\alpha \geq 0} f(x + \alpha d).$$

The notation $\arg \min$ (“argument of the minimum”) means: find the value of α (alpha) that makes the expression $f(x + \alpha d)$ as small as possible, subject to $\alpha \geq 0$ (we do not want to step backwards).

Line Search: Finding the Best Step Along a Direction II

Notation

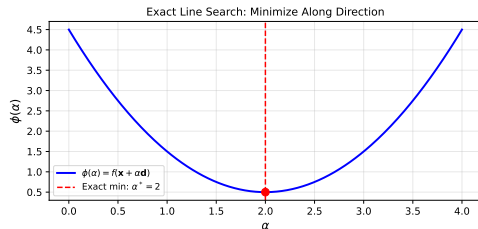
- α^* (alpha-star) — the optimal step size; the unique minimiser of the one-dimensional function $\phi(\alpha) = f(\mathbf{x} + \alpha \mathbf{d})$.
- $\phi(\alpha)$ — a shorthand for the 1-D function we are minimising during line search; it depends on the fixed current point $\mathbf{x}$ and direction $\mathbf{d}$.
- The constraint $\alpha \geq 0$ ensures we move in the direction $\mathbf{d}$, not opposite to it.

Exact Line Search for Quadratic Functions

For a quadratic objective $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{H} \mathbf{x} - \mathbf{g}^\top \mathbf{x}$, with steepest descent direction $\mathbf{d} = -\nabla f$, the optimal step has a closed form:

$$\alpha^* = \frac{\mathbf{d}^\top \mathbf{d}}{\mathbf{d}^\top \mathbf{H} \mathbf{d}}.$$

Here $\mathbf{H}$ (bold H) is the *Hessian matrix* — the matrix of second partial derivatives of f — and the two dot products measure the squared length of $\mathbf{d}$ and the



Line Search: Finding the Best Step Along a Direction III

How to Read the Quadratic Step Formula

The numerator $d^T d = \|d\|^2$ is the squared length of the descent direction. The denominator $d^T H d$ measures how much the function curves along d (the curvature). When the curvature is high (the function is sharply curved), the optimal step is small. When the curvature is low (the function is nearly flat), the optimal step is large. This makes intuitive sense: on a flat slope you should take big steps; on a tight curve you should take small ones to avoid overshooting.

Line Search: Finding the Best Step Along a Direction IV

Worked Example: Exact Line Search for a Quadratic

Let $f(x_1, x_2) = x_1^2 + x_1x_2 + x_2^2$. Starting at $\mathbf{x} = [3, 2]^\top$:

- 1 Gradient: $\nabla f = [2x_1 + x_2, x_1 + 2x_2]^\top = [8, 7]^\top$.
- 2 Descent direction: $\mathbf{d} = -\nabla f = [-8, -7]^\top$.
- 3 Hessian: $\mathbf{H} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$.
- 4 Numerator: $\mathbf{d}^\top \mathbf{d} = 64 + 49 = 113$.
- 5 Denominator: $\mathbf{d}^\top \mathbf{H} \mathbf{d} = [-8, -7] \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} [-8, -7]^\top = [-8, -7] [-23, -22]^\top = 184 + 154 = 338$.
- 6 Optimal step: $\alpha^* = 113/338 \approx 0.334$.
- 7 New point: $\mathbf{x}' = [3, 2] + 0.334 \times [-8, -7] \approx [0.33, -0.34]$.

Exact Line Search in Python (using SciPy)

```
1  import numpy as np
2  from scipy.optimize import minimize_scalar
3
4  def exact_line_search(f, x, d):
5      """
6      Minimize the 1-D function  $\phi(\alpha) = f(x + \alpha * d)$ 
7      over  $\alpha \geq 0$ . Returns optimal step size  $\alpha^*$ .
8      """
9      phi = lambda alpha: f(x + alpha * d)
10     result = minimize_scalar(phi, bounds=(0, 10), method='bounded')
11     return result.x
12
13     # -- Example:  $f(x_1, x_2) = x_1^2 + x_1 * x_2 + x_2^2$  -----
14     def f(x):
15         return x[0]**2 + x[0]*x[1] + x[1]**2
16
17     def grad_f(x):
18         return np.array([2*x[0] + x[1],      # partial derivative w.r.t.  $x_1$ 
19                          x[0] + 2*x[1]])    # partial derivative w.r.t.  $x_2$ 
20
21     x = np.array([3.0, 2.0])
22     d = -grad_f(x)                # steepest descent direction
23     alpha_star = exact_line_search(f, x, d)
24     x_new = x + alpha_star * d
25
26     print(f"Gradient at x:    {grad_f(x)}")
27     print(f"Descent dir d:    {d}")
```


Backtracking Line Search: A Cheaper Alternative I

Exact line search guarantees the best possible step in a given direction, but evaluating f many times along the ray is expensive when f is costly (e.g. running a simulation). *Backtracking line search* is a much cheaper alternative that merely asks: is this step *good enough*?

The “good enough” criterion is the *Armijo condition* (also called the first Wolfe condition or the sufficient decrease condition). It requires that the function value drops by at least a fixed fraction of what a linear model would predict:

Armijo (Sufficient Decrease) Condition

$$f(x + \alpha d) \leq f(x) + \beta \alpha \nabla f(x)^\top d.$$

Backtracking Line Search: A Cheaper Alternative II

Notation

- $f(x + \alpha d)$ — the actual function value at the proposed new point.
- $f(x) + \beta \alpha \nabla f(x)^\top d$ — a linear prediction of how much f should decrease, scaled by β (beta, the second Greek letter, pronounced “bay-ta”).
- β (beta) — a small positive constant, typically $\beta = 10^{-4}$. It sets how much decrease we require: $\beta = 0$ would accept any decrease; β close to 1 would require almost the full predicted decrease.
- $\nabla f(x)^\top d$ — the directional derivative at x along d ; this is negative (since d is a descent direction), so the right-hand side is *less than* $f(x)$.

Backtracking Line Search: A Cheaper Alternative III

How to Read the Armijo Condition

The right-hand side of the inequality is a line that starts at $f(x)$ when $\alpha = 0$ and slopes downward (since $\nabla f^\top d < 0$). The slope is scaled by $\beta < 1$, so it decreases more slowly than the tangent to $\phi(\alpha) = f(x + \alpha d)$. The condition says: the actual function value must lie *below* this gentle downward-sloping line. Intuitively, we require a *sufficient* decrease — not just any decrease.

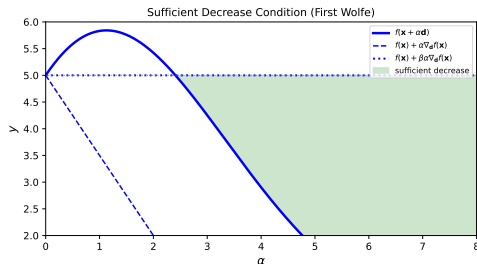
The backtracking algorithm starts with a large trial step α (often $\alpha = 1$) and repeatedly reduces it by a factor $p \in (0, 1)$ (p , the reduction factor, typically $p = 0.5$) until the Armijo condition is satisfied.

Backtracking Algorithm

- 1 Compute $y = f(x)$ and $g = \nabla f(x)$.
- 2 **While** $f(x + \alpha d) > y + \beta \alpha (g^\top d)$:

$$\alpha \leftarrow p \alpha.$$

- 3 **Return** α .



Numerical Example: Backtracking Line Search (Example 4.2) I

Let us trace through the backtracking algorithm step by step on a concrete example so there is no ambiguity about how the computation proceeds.

Setup

- Objective: $f(x_1, x_2) = x_1^2 + x_1x_2 + x_2^2$.
- Current point: $x = [1, 2]^\top$.
- Descent direction: $d = [-1, -1]^\top$.
- Parameters: $\alpha_{\max} = 10$, reduction factor $\rho = 0.5$, Armijo parameter $\beta = 10^{-4}$.

First, compute the quantities that stay constant throughout the search:

$$f(x) = 1^2 + 1 \cdot 2 + 2^2 = 1 + 2 + 4 = 7,$$

$$\nabla f(x) = [2x_1 + x_2, x_1 + 2x_2]^\top = [2 \cdot 1 + 2, 1 + 2 \cdot 2]^\top = [4, 5]^\top,$$

$$g^\top d = [4, 5] \cdot [-1, -1] = -4 - 5 = -9.$$

Numerical Example: Backtracking Line Search (Example 4.2) II

The threshold at step size α is:

$$y + \beta\alpha(\mathbf{g}^\top \mathbf{d}) = 7 + 10^{-4} \cdot \alpha \cdot (-9) = 7 - 0.0009\alpha.$$

Numerical Example: Backtracking Line Search (Example 4.2) III

Iteration 1: $\alpha = 10$

$$\begin{aligned} \mathbf{x} + \alpha \mathbf{d} &= [1, 2] + 10 \cdot [-1, -1] = [-9, -8], \\ f([-9, -8]) &= 81 + 72 + 64 = 217, \\ \text{Threshold} &= 7 - 0.009 = 6.991. \\ 217 > 6.991 &\Rightarrow \text{FAIL. } \alpha \leftarrow 5. \end{aligned}$$

Iteration 2: $\alpha = 5$

$$\begin{aligned} \mathbf{x} + \alpha \mathbf{d} &= [1, 2] + 5 \cdot [-1, -1] = [-4, -3], \\ f([-4, -3]) &= 16 + 12 + 9 = 37, \\ \text{Threshold} &= 7 - 0.0045 = 6.9955. \\ 37 > 6.9955 &\Rightarrow \text{FAIL. } \alpha \leftarrow 2.5. \end{aligned}$$

Numerical Example: Backtracking Line Search (Example 4.2) IV

Iteration 3: $\alpha = 2.5$

$$\begin{aligned} \mathbf{x} + \alpha \mathbf{d} &= [1, 2] + 2.5 \cdot [-1, -1] = [-1.5, -0.5], \\ f([-1.5, -0.5]) &= 2.25 + 0.75 + 0.25 = 3.25, \\ \text{Threshold} &= 7 - 0.00225 = 6.99775. \\ 3.25 &\leq 6.998 \quad \Rightarrow \quad \mathbf{ACCEPT.} \quad \alpha^* = 2.5. \end{aligned}$$

Summary

After only 3 halvings the algorithm accepts $\alpha^* = 2.5$, moving the design point from $[1, 2]$ to $[-1.5, -0.5]$ and reducing f from 7 to 3.25. Three function evaluations — much cheaper than a full 1-D optimisation.

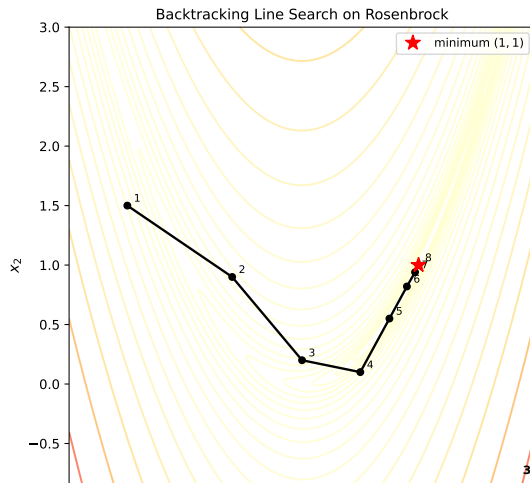
Backtracking on the Rosenbrock Function I

The Rosenbrock function is a classic test problem with a curved, narrow valley that is hard to navigate. The figure below shows the path traced by backtracking line search with steepest descent on this function, illustrating how the algorithm follows the valley floor.

Algorithm 4.3 Parameters

Symbol	Meaning
α	initial (maximum) step size
p	reduction factor, e.g. $p = 0.5$
β	Armijo parameter, e.g. $\beta = 10^{-4}$

At each outer iteration the algorithm evaluates f at $x + \alpha d$, $x + p\alpha d$, $x + p^2\alpha d$, and so on, until the Armijo condition is satisfied. In the best case only one evaluation is needed; in the worst case a few dozen.



Backtracking Line Search in Python

```
1  import numpy as np
2
3  def backtracking_line_search(f, grad_f, x, d,
4                              alpha=1.0, p=0.5, beta=1e-4):
5      """
6      Backtracking line search enforcing the Armijo condition.
7      Reduces alpha by factor p until sufficient decrease is satisfied.
8
9      Parameters
10     -----
11     f          : objective function
12     grad_f     : gradient function
13     x          : current design point
14     d          : descent direction (must satisfy grad_f(x) @ d < 0)
15     alpha      : initial (maximum) step size (default 1.0)
16     p          : reduction factor in (0, 1) (default 0.5)
17     beta       : Armijo parameter in (0, 1) (default 1e-4)
18     """
19     y          = f(x)                # current function value
20     g          = grad_f(x)
21     slope      = g @ d               # directional derivative; must be < 0
22
23     while f(x + alpha * d) > y + beta * alpha * slope:
24         alpha *= p                   # shrink step by factor p
25
26     return alpha
```

Why Backtracking Alone Is Not Enough I

The Armijo (backtracking) condition prevents steps that are too *large* — it ensures we decrease f by at least a fraction of the predicted decrease. However, it places *no lower bound* on the step size: a step of $\alpha = 10^{-100}$ would satisfy the Armijo condition trivially (any tiny decrease counts), yet it would make no practical progress.

We therefore need a *second condition* that prevents steps from being too small. The *Wolfe conditions* combine the Armijo condition with a curvature condition to give a pair of requirements that together guarantee the step is both “not too big” and “not too small”.

Why Backtracking Alone Is Not Enough II

The Two Wolfe Conditions

First Wolfe Condition (Armijo / sufficient decrease):

$$f(\mathbf{x} + \alpha \mathbf{d}) \leq f(\mathbf{x}) + \beta \alpha \nabla f(\mathbf{x})^\top \mathbf{d}.$$

Second Wolfe Condition (curvature condition):

$$\nabla f(\mathbf{x} + \alpha \mathbf{d})^\top \mathbf{d} \geq \sigma \nabla f(\mathbf{x})^\top \mathbf{d}.$$

The parameters satisfy $0 < \beta < \sigma < 1$, with typical values $\beta = 10^{-4}$ and $\sigma = 0.9$.

Why Backtracking Alone Is Not Enough III

Notation

- σ (sigma, the eighteenth Greek letter, pronounced “sig-ma”) — the curvature parameter; controls how tight the curvature condition is.
- $\nabla f(x + \alpha d)^\top d$ — the directional derivative at the *new* point $x + \alpha d$ along direction d ; it is the slope of $\phi(\alpha)$ at the proposed step.
- $\nabla f(x)^\top d$ — the directional derivative at the *current* point; it is the initial slope (negative, since d is a descent direction).

Why Backtracking Alone Is Not Enough IV

How to Read the Curvature Condition

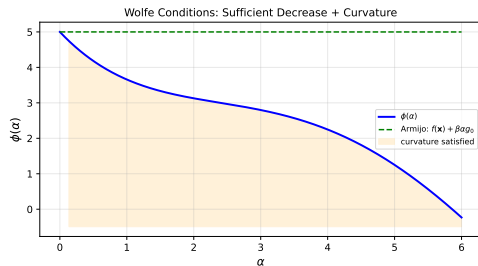
The second Wolfe condition says that the slope of $\phi(\alpha)$ at the accepted step must be *less negative* than σ times the initial slope. In other words, the function must have “flattened out” somewhat by the time we stop. If the slope is still very steep (very negative) at the accepted point, the step is too short and we could profitably have gone further. Setting σ close to 1 is lenient (allows nearly any step); setting σ close to β is strict (nearly exact line search).

Strong Wolfe Conditions

The *strong* version of the curvature condition uses an absolute value:

$$|\nabla f(x + \alpha d)^\top d| \leq \sigma |\nabla f(x)^\top d|.$$

This prevents accepting a step where the slope has merely changed sign (passed through a local maximum of ϕ), which the weak condition would allow. The strong Wolfe conditions are required by conjugate gradient and



The Strong Backtracking Algorithm I

Finding a step that satisfies the *strong* Wolfe conditions requires a more sophisticated procedure than simple backtracking. The standard approach proceeds in two phases.

Phase 1 — Bracket Phase: Find a Bracketing Interval

Start from $\alpha = 0$ and double the trial step until one of three *bracketing conditions* is triggered, which guarantees that a qualifying step exists somewhere in the bracket $[\alpha_{\text{prev}}, \alpha]$:

- 1 $f(x + \alpha d) > f(x) + \beta \alpha g_0$ (Armijo violated — overshoot).
- 2 $f(x + \alpha d) \geq f_{\text{prev}}$ (function increased — passed a local minimum).
- 3 $\nabla f(x + \alpha d)^\top d \geq 0$ (gradient changed sign — passed through a minimum of ϕ).

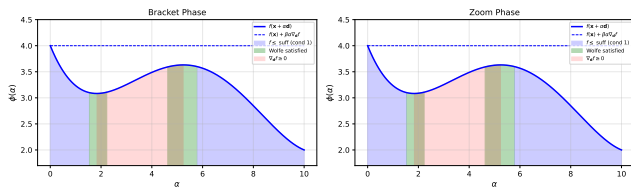
Here $g_0 = \nabla f(x)^\top d$ (the initial directional derivative, which is negative) and f_{prev} is the function value at the previous trial step.

The Strong Backtracking Algorithm II

Phase 2 — Zoom Phase: Narrow Down the Bracket

With the bracket $[\alpha_{lo}, \alpha_{hi}]$ in hand, bisect the interval repeatedly. At each bisection step, the bracket is updated so that it always contains a qualifying step. This is guaranteed to terminate with a step satisfying the strong Wolfe conditions.

Strong Backtracking Line Search



The left panel shows the bracket phase doubling α until a bracketing condition fires. The right panel shows the zoom phase bisecting the bracket. Each bisection halves the interval, giving rapid convergence to a qualifying step.

The Strong Backtracking Algorithm III

Bracketing Guarantee (Theorem)

An interval $[0, a]$ is guaranteed to contain a step satisfying the strong Wolfe conditions whenever any of the following holds:

- 1 $f(x + ad) \geq f(x)$ (function has not decreased).
- 2 $f(x + ad) > f(x) + \beta a \nabla_d f(x)$ (Armijo violated).
- 3 $\nabla_d f(x + ad) \geq 0$ (slope has become non-negative).

This theorem underpins the correctness of the strong backtracking algorithm: as soon as any of these conditions fires, we know a good step must lie in the current bracket.

Key Insight

The Wolfe conditions are the gold standard for line searches in quasi-Newton methods such as BFGS and L-BFGS. Satisfying them guarantees that the Hessian approximation matrices remain positive definite (so the Newton-like direction remains a descent direction), a property that backtracking alone cannot guarantee.

Strong Backtracking Line Search in Python

```
1 import numpy as np
2
3 def strong_backtracking(f, grad_f, x, d,
4                       alpha=1.0, beta=1e-4, sigma=0.1):
5     """Strong backtracking: finds alpha satisfying strong Wolfe conditions.
6
7     Phase 1 (bracket): expand alpha until a bracketing condition fires.
8     Phase 2 (zoom):    bisect the bracket until strong Wolfe is met.
9     """
10    y0 = f(x)
11    g0 = grad_f(x) @ d          # initial directional derivative (< 0)
12    y_prev, alpha_prev = float('nan'), 0.0
13    alpha_lo = alpha_hi = float('nan')
14
15    # -- Phase 1: Bracket -----
16    while True:
17        y = f(x + alpha * d)
18        # Bracketing condition: Armijo violated or function increased
19        if y > y0 + beta * alpha * g0 or \
20            (not np.isnan(y_prev) and y >= y_prev):
21            alpha_lo, alpha_hi = alpha_prev, alpha
22            break
23        g = grad_f(x + alpha * d) @ d
24        if abs(g) <= -sigma * g0:    # strong Wolfe curvature satisfied
25            return alpha
26        if g >= 0:                  # slope non-negative: bracket found
27            alpha_lo, alpha_hi = alpha, alpha_prev
```

Trust Region: A Different Philosophy I

Line search methods first choose a direction and then determine how far to step in that direction. Trust region methods flip this strategy: they choose *both* the direction and the step length simultaneously by solving a constrained subproblem within a local region where the model is trusted to be accurate.

The Core Idea

At the current point $\mathbf{x}^{(k)}$, build a simple (quadratic) model $\hat{f}$ of the objective function, then minimise that model within a ball of radius δ (delta, the fourth Greek letter, pronounced “del-ta”) centred at $\mathbf{x}^{(k)}$:

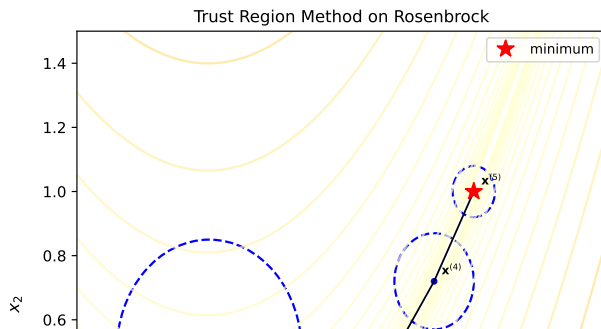
$$\hat{f}(\mathbf{x}') \approx f(\mathbf{x}^{(k)}) + \nabla f(\mathbf{x}^{(k)})^\top (\mathbf{x}' - \mathbf{x}^{(k)}) + \frac{1}{2}(\mathbf{x}' - \mathbf{x}^{(k)})^\top \mathbf{H}(\mathbf{x}' - \mathbf{x}^{(k)}).$$

The ball $\{\mathbf{x}' : \|\mathbf{x}' - \mathbf{x}^{(k)}\| \leq \delta\}$ is the *trust region* — the region within which we trust the model to be a good approximation to f .

Trust Region: A Different Philosophy II

Notation

- x' (x-prime) — the candidate next point (the variable we optimise over in the subproblem).
- $\hat{f}(x')$ (f-hat of x-prime) — the quadratic model approximating f near $x^{(k)}$.
- H (bold H) — the Hessian matrix (matrix of second partial derivatives) of f at $x^{(k)}$; or a quasi-Newton approximation thereof.
- δ (delta) — the trust region radius; a positive scalar that is adapted at each iteration.



The circle in the figure is the trust region boundary. Inside the circle the quadratic model (smooth curved surface) is a reliable approximation to the true function. The algorithm finds the minimum of the model inside the circle and uses that as the next iterate.

Trust Region: A Different Philosophy III

Trust Region Subproblem (Eq. 4.14)

$$\mathbf{x}^{(k+1)} = \arg \min_{\mathbf{x}'} \left[f(\mathbf{x}^{(k)}) + \nabla f(\mathbf{x}^{(k)})^\top (\mathbf{x}' - \mathbf{x}^{(k)}) + \frac{1}{2} (\mathbf{x}' - \mathbf{x}^{(k)})^\top \mathbf{H}(\mathbf{x}' - \mathbf{x}^{(k)}) \right] \quad \text{subject to } \|\mathbf{x}' - \mathbf{x}^{(k)}\| \leq \delta.$$

This is a *quadratic program with a quadratic constraint* (a spherical constraint). It can be solved efficiently in $O(n^3)$ time using eigenvalue methods or iterative solvers.

Contrast with Line Search

Line search: pick direction $\mathbf{d}$, then optimise $f(\mathbf{x} + \alpha \mathbf{d})$ over α . Trust region: pick the step $\mathbf{x}' - \mathbf{x}^{(k)}$ that minimises the model, subject to the step length not exceeding δ . The direction and step size emerge *together* from the subproblem. When δ is large the unconstrained Newton step may be taken; when δ is small the gradient descent step is taken. The trust region radius thus interpolates between gradient descent and Newton's method automatically.

Adapting the Trust Region Radius I

The trust region radius δ must be managed carefully: too small and we make tiny steps; too large and the quadratic model becomes inaccurate. The algorithm adapts δ by measuring how well the model predicted the actual decrease.

Improvement Ratio η (Eq. 4.15)

$$\eta = \frac{f(x^{(k)}) - f(x')}{f(x^{(k)}) - \hat{f}(x')} = \frac{\Delta f_{\text{actual}}}{\Delta f_{\text{predicted}}}.$$

This ratio (eta, the seventh Greek letter, pronounced “ay-ta”) compares the actual decrease in f to the decrease predicted by the quadratic model.

Adapting the Trust Region Radius II

Notation

- η (eta) — the improvement ratio; a dimensionless number comparing model quality to actual progress.
- $f(x^{(k)}) - f(x')$ — the *actual* decrease in the objective function; positive means the step helped.
- $f(x^{(k)}) - \hat{f}(x')$ — the *predicted* decrease according to the quadratic model; this is always positive (the model always predicts improvement from its own minimum).

Adapting the Trust Region Radius III

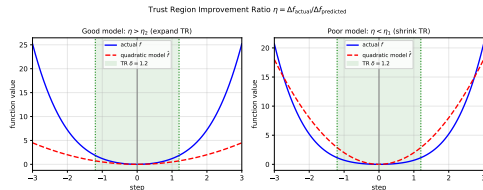
How to Read the Improvement Ratio

- $\eta \approx 1$: the model is excellent; the actual and predicted decreases nearly match. Expand the trust region ($\delta \leftarrow \gamma_2 \delta$, gamma-two times delta) so future steps can be bolder.
- $\eta_1 \leq \eta \leq \eta_2$: the model is adequate; keep δ unchanged.
- $\eta < \eta_1$: the model is poor (or the actual decrease was far less than predicted). Shrink the trust region ($\delta \leftarrow \gamma_1 \delta$) and reject the step.
- $\eta < 0$: the function actually *increased* (bad step). Reject the step and shrink δ substantially.

Trust Region Update Rules

Typical parameters: $\eta_1 = 0.25$, $\eta_2 = 0.75$, $\gamma_1 = 0.5$, $\gamma_2 = 2.0$.

- If $\eta < \eta_1$: **reject** the step; set $\delta \leftarrow \gamma_1 \delta$.
- If $\eta_1 \leq \eta \leq \eta_2$: **accept** the step; keep δ .
- If $\eta > \eta_2$: **accept** the step; set $\delta \leftarrow \gamma_2 \delta$.



Adapting the Trust Region Radius IV

Elliptical Trust Region (Eq. 4.16)

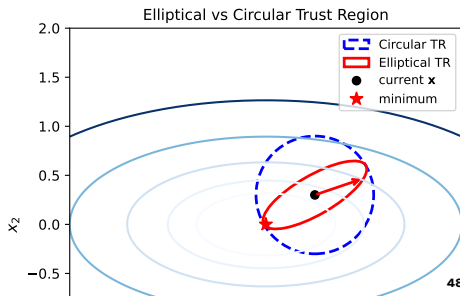
The circular trust region can be generalised to an *elliptical* trust region:

$$\|x - x_0\|_E^2 = (x - x_0)^T E (x - x_0) \leq 1,$$

where E (bold E) is a positive-definite matrix that defines the shape of the ellipse. By elongating the trust region along directions of high curvature confidence, an elliptical region can model the actual function more accurately than a sphere.

Notation

- E — the ellipse-shaping matrix; a positive-definite symmetric matrix. When $E = I$ (the identity matrix) the ellipse is a circle.
- $\|x\|_E^2 = x^T E x$ — the E -weighted norm (“energy norm”); it measures length in a geometry distorted by E .



Trust Region Descent in Python (using SciPy)

```
1 import numpy as np
2 from scipy.optimize import minimize
3
4 def trust_region_step(grad, hess, delta):
5     """
6     Solve the circular trust region subproblem:
7         min grad @ s + 0.5 * s @ hess @ s
8         s.t. ||s|| <= delta
9     Returns the step vector s.
10    """
11    n = len(grad)
12    def model(s): return grad @ s + 0.5 * s @ hess @ s
13    def model_grad(s): return grad + hess @ s
14    constraints = {'type': 'ineq',
15                  'fun': lambda s: delta - np.linalg.norm(s)}
16    res = minimize(model, np.zeros(n), jac=model_grad,
17                  constraints=constraints, method='SLSQP')
18    return res.x
19
20 def trust_region_descent(f, grad_f, hess_f, x0, delta=1.0,
21                          eta1=0.25, eta2=0.75,
22                          gamma1=0.5, gamma2=2.0, max_iter=100):
23     """Trust region method with adaptive radius delta."""
24     x = x0.copy().astype(float)
25     for _ in range(max_iter):
26         g = grad_f(x)
27         H = hess_f(x)
```

When Should We Stop Iterating? I

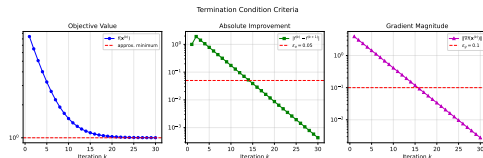
Every iterative algorithm needs a stopping criterion. Stopping too early wastes the accuracy we could have achieved; stopping too late wastes computation (and sometimes causes numerical instability as we try to squeeze out the last few decimal places).

There are four standard termination conditions for descent methods. In practice, good implementations check *all four* and stop as soon as *any* of them is triggered.

Condition 1: Maximum Iterations (Eq. 4.17)

$$k > k_{\max}.$$

Here k (k) is the current iteration number and $k_{\max}$ (k -sub-max) is a user-specified upper limit, e.g. $k_{\max} = 10\,000$. This is a *safety net*: the algorithm always terminates in finite time regardless of whether it has converged.



Condition 2: Absolute Improvement (Eq. 4.18)

$$f(x^{(k)}) - f(x^{(k+1)}) < \epsilon_f.$$

When Should We Stop Iterating? II

Condition 3: Relative Improvement (Eq. 4.19)

$$f(x^{(k)}) - f(x^{(k+1)}) < \epsilon_r |f(x^{(k)})|.$$

Here ϵ_r (epsilon sub r) is the relative tolerance, e.g. $\epsilon_r = 10^{-6}$. The condition fires when the improvement is small *relative to the current function value*. This is important when f takes very large values: an absolute improvement of 10^{-8} on a function with value 10^6 may well be worth continuing. The relative criterion is *scale-invariant*: it automatically adapts to the magnitude of f .

Condition 4: Gradient Magnitude (Eq. 4.20)

$$\|\nabla f(x^{(k+1)})\| < \epsilon_g.$$

Here ϵ_g (epsilon sub g) is the gradient tolerance, e.g. $\epsilon_g = 10^{-6}$, and $\|\cdot\|$ denotes the Euclidean norm (length of a vector). This condition is the most theoretically principled: at a local minimum the gradient is exactly zero, so a small gradient is direct evidence of convergence.

When Should We Stop Iterating? III

How to Read the Gradient Condition

The gradient $\nabla f(x)$ is the slope of the surface. When the slope is nearly zero the surface is nearly flat, meaning we are near a critical point (possibly a minimum, a maximum, or a saddle point). Combined with the fact that f has been decreasing at each step, a small gradient strongly suggests we have reached a local minimum.

When Should We Stop Iterating? IV

Why Multiple Conditions?

Each individual condition has a failure mode:

- *Max iterations only*: may stop too early (before convergence) or too late (after convergence, wasting time).
- *Gradient only*: fails on flat functions or very slowly converging sequences where $\|\nabla f\|$ stays above ϵ_g for a very long time.
- *Absolute improvement only*: fails when f is very large (a tiny relative change gives a large absolute change) or when the algorithm makes slow but steady progress.
- *Relative improvement only*: can terminate too early when f is near zero (any improvement looks small relative to a tiny value).

Using all four conditions simultaneously handles all failure modes. A robust implementation typically uses: gradient + relative improvement + max iterations as the primary safety net.

When Should We Stop Iterating? V

Typical Parameter Values

Condition	Symbol	Typical value
Max iterations	$k_{\max}$	10^3 – 10^5
Abs. improvement	ϵ_a	10^{-8}
Rel. improvement	ϵ_r	10^{-6}
Gradient norm	ϵ_g	10^{-6}

Termination Conditions in Python

```
1  import numpy as np
2
3  def check_termination(x_prev, x_curr, f_prev, f_curr, grad_curr,
4                        k, k_max=10000,
5                        eps_a=1e-8, eps_r=1e-6, eps_g=1e-6):
6      """
7      Check all four standard termination conditions.
8
9      Parameters
10     -----
11     x_prev, x_curr : design points at iterations k and k+1
12     f_prev, f_curr : function values at those points
13     grad_curr      : gradient at x_curr
14     k              : current iteration number
15     k_max          : max iterations (safety net)
16     eps_a          : absolute improvement tolerance
17     eps_r          : relative improvement tolerance
18     eps_g          : gradient-norm tolerance
19
20     Returns
21     -----
22     (terminated: bool, reason: str)
23     """
24     # Condition 1: maximum iterations
25     if k >= k_max:
26         return True, f"max iterations ({k_max}) reached"
```

Chapter 4 Summary I

This chapter introduced the general framework of *local descent* and the key design choices involved in making it work well in practice. Here is a concise recap of the main ideas.

Descent Direction Iteration

Every method in this chapter is a special case of the four-step template:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{d}^{(k)},$$

where $\mathbf{d}^{(k)}$ is a descent direction (satisfying $\nabla f(\mathbf{x}^{(k)})^\top \mathbf{d}^{(k)} < 0$) and $\alpha^{(k)}$ is a step factor chosen to ensure sufficient decrease.

Step Factors

The step factor $\alpha^{(k)}$ (alpha) can be:

- **Fixed:** constant α_0 — simple but fragile.
- **Exponentially decaying:** $\alpha^{(k)} = \gamma^k \alpha^{(0)}$ — bounded total distance; γ (gamma) controls speed of decay.
- **Polynomially decaying:** $\alpha^{(k)} = \alpha^{(0)} / (1 + \beta k)^p$ — unbounded distance when $p \leq 1$; needed for stochastic methods.
- **Chosen by line search:** most principled but more costly.

Line Search Methods

- **Exact line search:** minimises f along d exactly. Costly; has a closed form for quadratic f .
- **Backtracking (Armijo):** reduces α by factor p (p) until the sufficient-decrease condition $f(x + \alpha d) \leq f(x) + \beta \alpha \nabla f(x)^\top d$ is satisfied. Cheap; no curvature guarantee.
- **Wolfe conditions:** add a curvature condition (second Wolfe) to prevent steps from being too small. The *strong* Wolfe conditions use an absolute value on the curvature check, required by quasi-Newton methods.
- **Strong backtracking:** two-phase algorithm (bracket + zoom) that efficiently finds a step satisfying the strong Wolfe conditions.

Chapter 4 Summary IV

Trust Region Methods

Trust region methods simultaneously choose direction and step length by solving a quadratic subproblem within a ball of radius δ (delta). The radius is adapted at each iteration via the improvement ratio η (eta) = actual decrease / predicted decrease. A high η means the model is good and δ can be expanded; a low η means the model is poor and δ must shrink.

Termination Conditions

Use at least two of the four conditions (max iterations, absolute improvement ϵ_a , relative improvement ϵ_r , gradient norm ϵ_g) to ensure robust and efficient stopping. Each condition alone has a failure mode; combining them covers all cases.

The Big Picture

Local descent is the foundation of virtually all practical optimisation algorithms. The differences between methods — gradient descent, Newton, quasi-Newton, conjugate gradient — arise entirely from different choices of the descent direction $d^{(k)}$. The step factor selection strategies (fixed, decaying, line search, trust region) apply equally to all of them.

Key Formulas at a Glance I

The following table collects every major formula from this chapter with its full notation explained.

Descent Update

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{d}^{(k)}$$

$\mathbf{x}^{(k)}$: current point; $\alpha^{(k)}$ (alpha): step size; $\mathbf{d}^{(k)}$: descent direction.

Trust Region Subproblem

$$\min_{\|\mathbf{x}' - \mathbf{x}\| \leq \delta} \hat{f}(\mathbf{x}')$$

δ (delta): trust region radius; $\hat{f}$: quadratic model of f .

Exponential Step Decay

$$\alpha^{(k)} = \gamma^k \alpha^{(0)}, \quad 0 < \gamma < 1$$

γ (gamma): decay rate; $\alpha^{(0)}$: initial step.

Improvement Ratio

$$\eta = \frac{f(\mathbf{x}) - f(\mathbf{x}')}{f(\mathbf{x}) - \hat{f}(\mathbf{x}')}$$

η (eta): improvement ratio; numerator: actual decrease; denominator: predicted decrease.

First Wolfe (Armijo)

$$f(\mathbf{x} + \alpha \mathbf{d}) \leq f(\mathbf{x}) + \beta \alpha \nabla_{\mathbf{d}} f(\mathbf{x})$$

β (beta): Armijo parameter, typically 10^{-4} ;

Termination: Gradient

Exercise 4.1: Failure Modes of Single Termination Conditions

Explain in detail why relying on the gradient magnitude condition $\|\nabla f\| < \epsilon_g$ alone can be insufficient. Give two specific scenarios: (a) a case where the gradient is small but the algorithm has not found a good solution, and (b) a case where the gradient remains large for many iterations even though the function value has essentially converged.

Exercise 4.2: First Wolfe Condition on a Sinusoid

Consider gradient descent on $f(x) = \sin(x)$ starting at $x^{(1)} = -2$. The gradient is $f'(x) = \cos(x)$.

- 1 Compute the steepest descent direction at $x^{(1)} = -2$.
- 2 Using the first Wolfe condition with $\beta = 0.1$, determine whether $\alpha = 0.5$ is an acceptable step.
- 3 What is the approximate maximum α that satisfies the Armijo condition?

Exercise 4.3: Exact Line Search for a Quadratic

For $f(x_1, x_2) = x_1^2 + 2x_2^2$, starting at $x = [1, 1]^\top$:

- 1 Compute the gradient $\nabla f(x)$ and verify that the steepest descent direction is $d = [-2, -4]^\top$.
- 2 Write down the Hessian matrix H of f .
- 3 Apply the quadratic line search formula $\alpha^* = d^\top d / (d^\top H d)$ to find the exact step size.
- 4 Compute the new point $x' = x + \alpha^* d$ and verify that $f(x') < f(x)$.

Exercise 4.4: Trust Region Radius Adaptation

Suppose the trust region algorithm produces the following data at one iteration:

$$f(x^{(k)}) = 10.0, \quad f(x') = 9.2, \quad \hat{f}(x') = 8.5.$$

Compute the improvement ratio η . With parameters $\eta_1 = 0.25$, $\eta_2 = 0.75$, $\gamma_1 = 0.5$, $\gamma_2 = 2.0$, and current radius $\delta = 1.0$:

- 1 Should the step be accepted or rejected?
- 2 What should the new trust region radius δ' be?

Further Reading I

The material in this chapter forms the foundation of a large body of numerical optimisation literature. The following references provide rigorous proofs and deeper treatments.

- J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. Springer, 2006. Chapters 2–4 cover descent direction iteration, line search, and trust region methods with full convergence proofs. This is the definitive graduate-level reference.
- A. Conn, N. Gould, and P. Toint, *Trust Region Methods*, SIAM, 2000. The authoritative reference for trust region algorithms, including the subproblem solvers and convergence theory.
- Y. Nesterov, *Introductory Lectures on Stochastic Optimization*, Springer, 2004. Covers polynomial step-decay schedules and their role in convergence of stochastic gradient methods.
- SciPy documentation:
 - `scipy.optimize.line_search_wolfe2` — Strong Wolfe line search implementation.
 - `scipy.optimize.minimize` with `method='trust-ncg'` or `method='trust-krylov'` — trust region methods for general nonlinear objectives.
 - `scipy.optimize.minimize_scalar` — Exact 1-D line search (golden section, bounded Brent).

End of Chapter 4